

Mobile Authentication & Biometric Implementation Report

Jaxon Doolittle

Georgia Southwestern State University

CSCI 6130: Mobile Security

Dr. Jonathan Hobbs

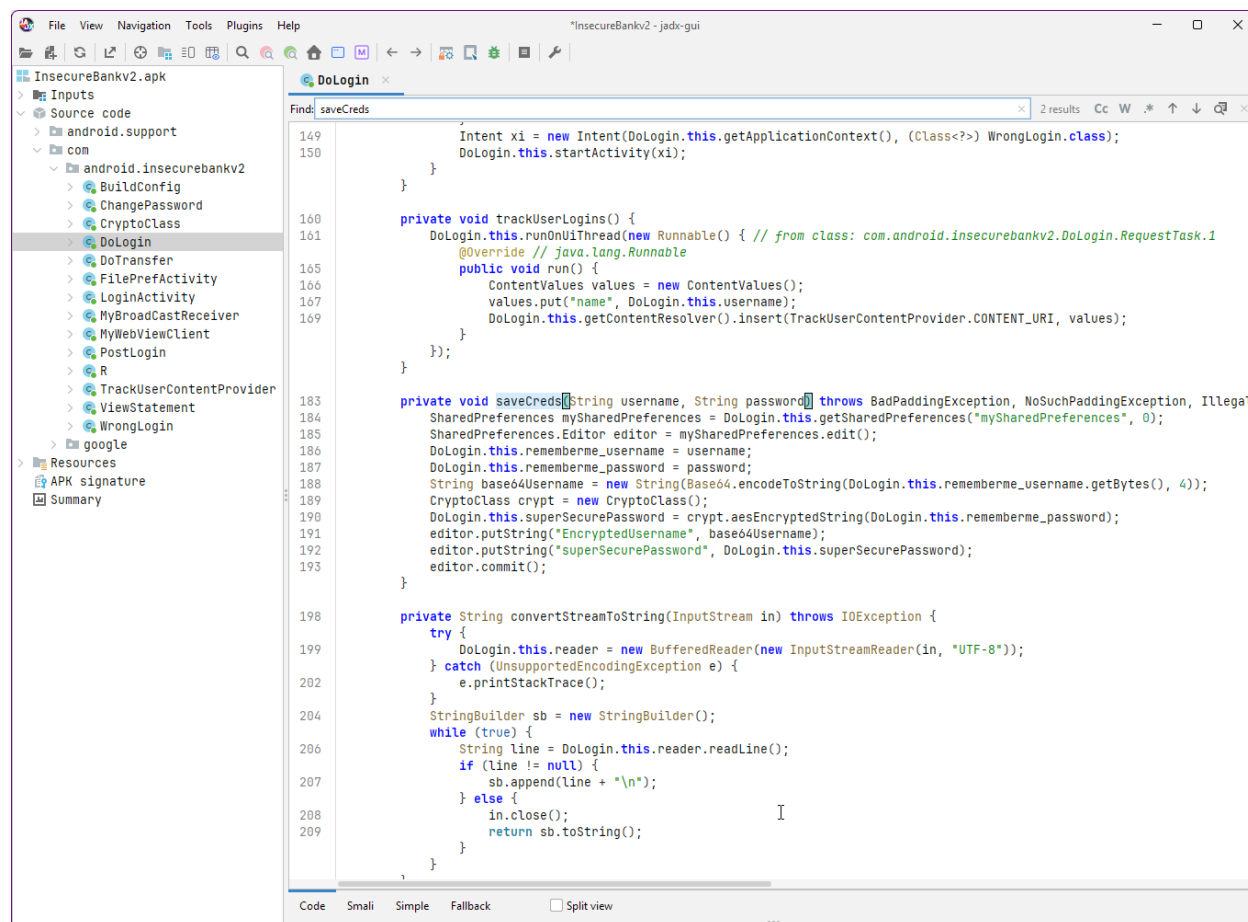
July 6, 2026

Part 1: Analysis Summary

To figure out how the OWASP InsecureBankv2 app handles user credentials, I loaded the APK into Jadx-GUI for static analysis. My goal was to see exactly what happens under the hood when a user logs in.

Finding: Insecure Token and Credential Storage. While looking through the decompiled Java code, I specifically analyzed the `com.android.insecurebankv2.DoLogin` class. I found a major flaw in how the app uses `SharedPreferences` inside the `saveCreds()` method.

When a user logs in, the app takes their username and password and saves them directly to a local XML file on the device. It runs the username through Base64 encoding, but Base64 is just an encoding format, it's not encryption, and anyone can decode it instantly. The password is passed through a custom AES encryption class, but because the developers didn't use the secure Android Keystore, the encryption keys are likely just hardcoded elsewhere in the app. This makes reversing the "encryption" trivial.



Summary of Authentication Flaws Observed: Because the developers didn't use the Android Keystore or EncryptedSharedPreferences, these credentials are essentially sitting in plain sight on the phone's file system. If an attacker gets physical access to the device, roots it, or uses another malicious app to exploit file permissions, they can easily extract the mySharedPreferences.xml file, decode the Base64 username, and reverse the poorly encrypted password, and steal the user's banking login.

Part 2: Implementation Activity (Biometric Login)

For the second part of the lab, I created a new Android Studio project to implement a secure biometric login flow using Android's BiometricPrompt API. This is a much better approach because it relies on the device's hardware security instead of saving passwords locally.

Steps taken to add and test biometric login:

1. **Gradle Setup:** I opened the build.gradle.kts file and added the required AndroidX biometric dependency so I could use the modern prompt API. Dependency added: implementation 'androidx.biometric:biometric:1.2.0-alpha04'
2. **Java Implementation:** In my MainActivity.java, I set up the BiometricPrompt to listen for the login button click. I configured the callbacks for when the authentication succeeds, fails, or throws an error.

Here is the core logic I used for the implementation:

```
package com.example.biometricauth;

import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.biometric.BiometricPrompt;
import androidx.core.content.ContextCompat;
import java.util.concurrent.Executor;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
```

```

Button loginButton = findViewById(R.id.btnLogin);

Executor executor = ContextCompat.getMainExecutor(this);
BiometricPrompt biometricPrompt = new BiometricPrompt(MainActivity.this,
    executor, new BiometricPrompt.AuthenticationCallback() {

        @Override
        public void onAuthenticationError(int errorCode, @NonNull CharSequence errString) {
            super.onAuthenticationError(errorCode, errString);
            Toast.makeText(getApplicationContext(), "Auth error: " + errString,
                Toast.LENGTH_SHORT).show();
        }

        @Override
        public void onAuthenticationSucceeded(@NonNull BiometricPrompt.AuthenticationResult
            result) {
            super.onAuthenticationSucceeded(result);
            Toast.makeText(getApplicationContext(), "Authentication Succeeded! Welcome.",
                Toast.LENGTH_SHORT).show();
        }

        @Override
        public void onAuthenticationFailed() {
            super.onAuthenticationFailed();
            Toast.makeText(getApplicationContext(), "Authentication failed",
                Toast.LENGTH_SHORT).show();
        }
    });

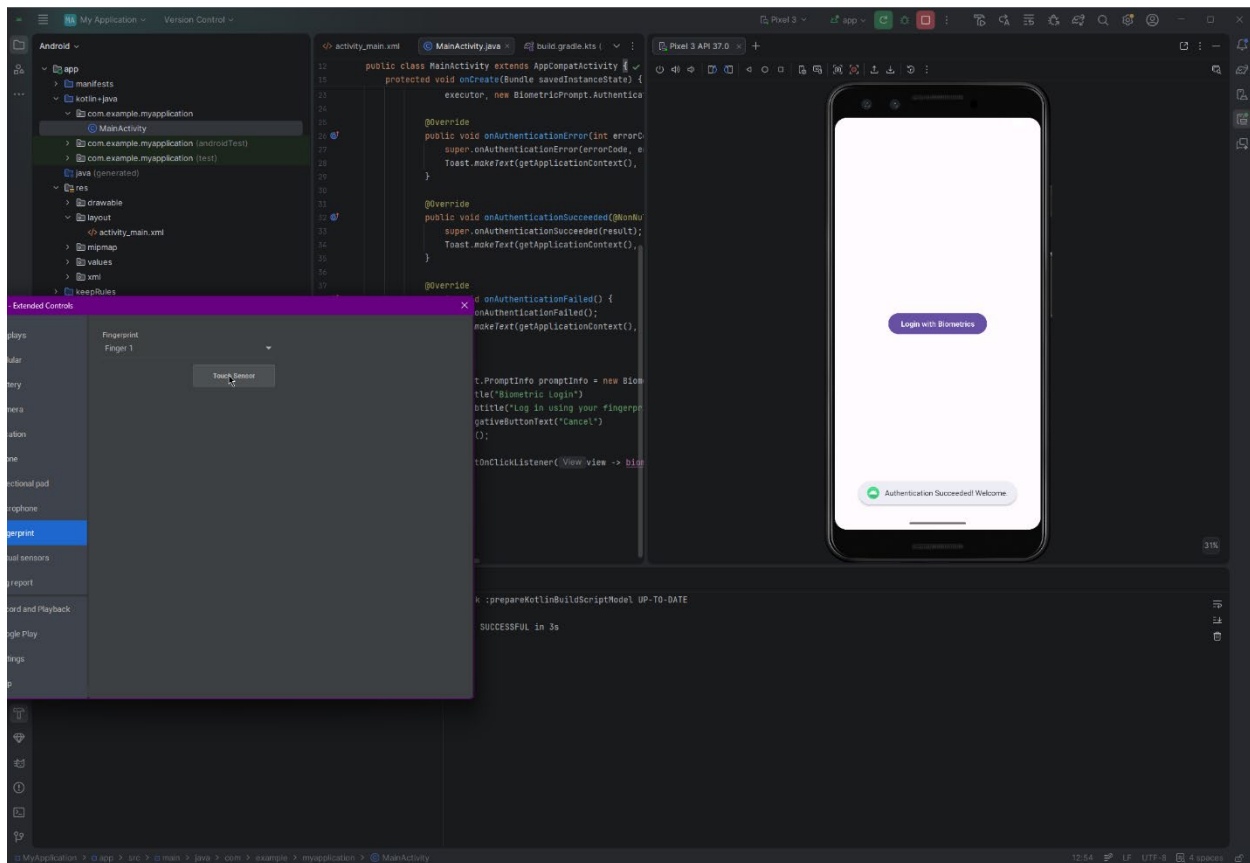
BiometricPrompt.PromptInfo promptInfo = new BiometricPrompt.PromptInfo.Builder()
    .setTitle("Biometric Login")
    .setSubtitle("Log in using your fingerprint credential")
    .setNegativeButtonText("Cancel")
    .build();

loginButton.setOnClickListener(view -> biometricPrompt.authenticate(promptInfo));
}

}

```

Emulator Testing: I ran the app on my Pixel 3 API 37 emulator. After clicking the login button, the biometric prompt popped up. I then opened the emulator's Extended Controls, navigated to the Fingerprint menu, and clicked "Touch Sensor" to simulate a successful fingerprint read.



Part 3: Security Reflection

Why is secure token handling critical in mobile? Having strong network security doesn't mean much if the data is stored insecurely at rest on the device. Mobile phones get lost, stolen, or compromised by malware all the time. If a developer stores a session token or a password without using the Keystore (exactly like I found in InsecureBankv2), an attacker can just bypass the network entirely. They can pull the raw credentials directly from the local file system and hijack the user's account without ever needing to intercept live web traffic.

Would your implementation pass a security audit? Even though my biometric implementation works and successfully brings up the system prompt, it would not pass a strict security audit for a financial app right now.

To actually pass a commercial audit, the biometric prompt needs to be cryptographically bound to a `CryptoObject`. Right now, my app just returns a simple boolean "Success" message when the fingerprint matches. A real security implementation requires the biometric authentication to unlock a specific cryptographic key safely stored inside the Android Keystore, which is then used

to decrypt the session token. Without that `CryptoObject` binding, a skilled attacker on a rooted device could potentially use a framework like Frida to hook the `onAuthenticationSucceeded` callback and bypass the physical hardware sensor entirely.